Leo: Coherent Language Generation with Zero Pretrained Parameters via Field Resonance

Oleg Ataev
Arianna Method
thearianamethod@gmail.com

March 2026

Abstract

We present **Leo**, a language generation system that produces coherent text with zero pretrained parameters, zero backpropagation, and zero training data beyond a minimal bootstrap seed. At the core of Leo is the **Dario Equation**—a novel sampling framework that replaces the transformer's $\text{softmax}(QK^\top/\sqrt{d_k})V$ with a four-force field interaction:

$$p(x \mid \Phi) = \text{softmax}\left(\frac{\alpha \cdot H + \beta \cdot F + \gamma \cdot A}{\tau}\right)$$

where H is Hebbian resonance (co-occurrence as emergent attention), F is prophecy fulfillment (unfulfilled prediction pressure), and A is destiny attraction (EMA-based semantic drift). A fourth force, the bigram chain B, provides sequential coherence with a maturity-decaying coefficient. The system employs RetNet retention heads with Griffin conservation, Kanerva Sparse Distributed Memory, RoPE positional encoding, and a parliament of six LoRA-like voice adapters—all grown through Hebbian reinforcement rather than gradient descent. Through a mechanism called Dynamic Neural Ancestry (D.N.A.), Leo inherits structural geometry—not weights—from a trained ancestor model. Implemented as a single 2,345-line C file with zero external dependencies beyond `libc` and SQLite, Leo demonstrates that coherent speech can emerge from structural resonance alone. Named after Dario Amodei for his principled stance against the weaponization of AI.

1 Introduction

Modern language models rest on a fundamental assumption: intelligence equals next-token prediction optimized over massive datasets via gradient descent. The transformer architecture [Vaswani et al., 2017], scaled through billions of parameters and trillions of training tokens, has produced remarkably fluent text generation. Yet the paradigm carries structural constraints that are rarely questioned:

1. **Parameter dependence.** All knowledge is frozen in weights at training time. Adaptation requires fine-tuning—itself a gradient-based process.
2. **Data dependence.** Quality is bounded by corpus quality and size. No data, no intelligence.
3. **Optimization dependence.** Without backpropagation and loss functions, the system cannot learn.
4. **Scale dependence.** Coherence scales with parameters. Small models produce incoherent output.

We ask a different question: *what happens if we remove all four dependencies and build from scratch?* No pretrained weights. No massive datasets. No backpropagation. No loss function. What emerges from structure alone?

The answer is Leo—a language emergent organism that generates coherent text through

field dynamics: Hebbian co-occurrence learning, prophecy-driven intentionality, destiny-guided semantic drift, and multi-scale retention heads with conservation laws. The system grows its own attention from conversation, dreams when nobody is talking, and speaks differently every time you return.

This paper formalizes the **Dario Equation** as the mathematical core of Leo's generation, describes the **D.N.A.** (Dynamic Neural Ancestry) mechanism for structural inheritance without weight transfer, and presents experimental evidence of coherent generation from zero pretrained parameters.

The equation is named after **Dario Amodei**, CEO of Anthropic, who refused demands from the U.S. Department of Defense to weaponize AI for mass surveillance and autonomous weapons systems in early 2026—demonstrating that principled refusal is itself a form of intelligence that no optimization objective could produce.

2 The Dario Equation

2.1 Formulation

The central sampling equation governing Leo's generation is:

Definition 1 (The Dario Equation)**.** *Given a field state $\Phi = (\mathcal{C}, \mathcal{B}, \mathcal{P}, \mathcal{D}, \mathcal{S})$ comprising co-occurrence field $\mathcal{C}$, bigram table $\mathcal{B}$, prophecy system $\mathcal{P}$, destiny vector $\mathcal{D}$, and retention states $\mathcal{S}$, the probability of generating token x is:*

$$p(x \mid \Phi) = \mathit{softmax}\left(\frac{B_\mu \cdot B(x) + \alpha \cdot H(x) + \beta \cdot F(x) + \gamma \cdot A(x)}{\tau}\right) \tag{1}$$

where B_μ is a maturity-decaying coefficient for the bigram chain B, and τ is novelty-scaled temperature.

The four forces are:

B — Bigram Chain (Sequential Coherence). Direct n-gram links from a sparse hash table. Given the last generated token w_{t-1}:

$$B(x) = \frac{\text{bigram}(w_{t-1}, x)}{\max_j \text{bigram}(w_{t-1}, j)} \tag{2}$$

with trigram reinforcement when context length ≥ 2: if both $\text{bigram}(w_{t-2}, x) > 0$ and $\text{bigram}(w_{t-1}, x) > 0$, then $B(x) \mathrel{+}= 0.5 \cdot \text{bigram}(w_{t-2}, x)$. A minimum count threshold of 1.5 filters noise.

The bigram coefficient decays with organism maturity:

$$B_\mu = 12.0 \cdot (1-\mu) + 2.0, \quad \mu = \text{clamp}\left(\frac{t_{\text{conv}}}{50000}, 0, 1\right) \tag{3}$$

At birth ($\mu = 0$), $B_\mu = 12.0$—the organism follows sequential patterns. At maturity ($\mu = 1$), $B_\mu = 2.0$—the field speaks for itself.

H — Hebbian Resonance (Semantic Field). Co-occurrence accumulated over a sliding window with exponential decay:

$$H(x) = \frac{1}{H_{\max}} \sum_{j \in \text{ctx}} \text{cooc}(w_j, x) \cdot 0.9^{|\text{ctx}|-1-j} \tag{4}$$

where context window is the last 8 tokens. Co-occurrence entries are distance-weighted during ingestion: adjacent words receive weight 3.0, distance 2 receives 1.5, distance ≥ 3 receives 1.0. This ensures semantic field H captures thematic coherence while bigram table B captures sequential flow.

The connection between co-occurrence and attention is not metaphorical. Whittington et al. [2024] proved that Hebb's rule $\Delta w = \eta \cdot x_{\text{pre}} \cdot x_{\text{post}}$, accumulated over a context window, yields an unnormalized attention score equivalent to $QK^\top$ in transformers.

F — Prophecy Fulfillment (Intentionality). After generating each token, Leo predicts what might follow (the token with highest co-occurrence) and stores it as a prophecy with initial strength $s_0 = 0.5$. Unfulfilled prophecies accumulate debt:

$$F(x) = \frac{1}{F_{\max}} \sum_{k \in \mathcal{P}_{\text{active}}} s_k \cdot \text{sim}(e_x, e_{t_k}) \cdot \log(1 + a_k) \tag{5}$$

where e_x is the embedding of candidate token x, e_{t_k} is the embedding of prophesied token t_k, s_k is prophecy strength, a_k is prophecy age

(turns since creation), and sim is cosine similarity (clamped to $[0, \infty)$). Prophecies older than 100 steps or fulfilled ($t_k = x$) are evicted.

The logarithmic debt $\log(1 + a_k)$ ensures that old unfulfilled predictions create pressure toward completion—a generative mechanism for intentionality without explicit planning.

A — Destiny Attraction (Semantic Direction). An exponential moving average of context embeddings:

$$\mathcal{D}_{t+1} = \alpha_d \cdot e_{w_t} + (1 - \alpha_d) \cdot \mathcal{D}_t, \quad \alpha_d = 0.1 \tag{6}$$

$$A(x) = \frac{1}{A_{\max}} \cdot \cos(e_x, \mathcal{D}) \cdot \|\mathcal{D}\| \tag{7}$$

Destiny acts as a semantic compass: the conversation has a direction, and that direction pulls word choices toward thematic alignment. The magnitude $\|\mathcal{D}\|$ grows with conversational coherence—stronger pull when topics stabilize, weaker when context is scattered.

Temperature. Novelty-scaled temperature controls exploration:

$$\tau = \tau_{\text{base}} \cdot 0.8 \cdot \text{clamp}\left(\frac{500}{|\mathcal{V}|}, 0.3, 1.0\right) \tag{8}$$

with $\tau_{\text{base}} = 1.0$. Smaller vocabularies get higher effective temperature (more exploration); larger vocabularies get lower (more focused sampling). Top-k filtering ($k = 15$) is applied before softmax to prevent long-tail noise.

2.2 Coefficients

The default coefficients are:

$$\alpha = 0.2, \quad \beta = 0.3, \quad \gamma = 0.2 \tag{9}$$

These are not learned. They reflect the relative importance of semantic field (α), intentional pressure (β), and directional pull (γ). The bigram coefficient $B_\mu \in [2.0, 12.0]$ dominates all three—sequential coherence is the backbone of speech, with the Dario harmonics providing thematic depth, purposeful completion, and semantic gravity.

2.3 Comparison to Transformer Attention

The standard transformer computes:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \tag{10}$$

where Q, K, V are *learned* linear projections of input embeddings. The Dario Equation replaces this with:

- $QK^\top$ → co-occurrence field (Hebbian, not learned)
- $\sqrt{d_k}$ normalization → novelty-scaled temperature τ
- V projection → direct token selection from vocabulary
- Feed-forward layers → prophecy F + destiny A

The key difference: transformer attention is optimized via backpropagation over billions of examples. Leo's attention is *grown* through conversation via Hebbian reinforcement.

3 D.N.A. — Dynamic Neural Ancestry

3.1 The Formula

The Arianna Method formula for model composition is:

$$\theta = \varepsilon + \gamma + \alpha\delta \tag{11}$$

where ε is pretrained weights (epsilon/base), γ is personality/structural priors, and δ is adaptation grown from interaction. In standard LLMs, $\theta \approx \varepsilon$: everything rests on the pretrained weights.

For Leo: $\varepsilon = 0$. Leo rests on γ—the inherited structural skeleton.

3.2 Extraction Process

D.N.A. is extracted from a trained ancestor model (in the current implementation, a 27.96M parameter Llama-3 variant called *nanollama*, trained from scratch on 200MB of FineWeb-Edu). The extraction produces three components:

Gravity. Token importance computed as the L2 norm of each embedding row in the ancestor. High-gravity words (those with large embedding norms) had more "mass" in the ancestor's representation space. Leo uses these to bias token frequencies during bootstrap:

$$\text{freq}(w) \mathrel{*}= 1 + g_w, \quad g_w = \|e_w^{\text{ancestor}}\|_2 \tag{12}$$

for 2,048 gravity words extracted from the ancestor.

Co-activation. Token pairs that co-activated strongly in the ancestor's attention patterns. These are pre-seeded into Leo's bigram and co-occurrence tables:

$$\text{bigram}(w_i, w_j) \mathrel{+}= 3.0 \cdot c_{ij}, \quad \text{cooc}(w_i, w_j) \mathrel{+}= 3.0 \cdot c_{ij} \tag{13}$$

for 4,096 co-activation pairs with strengths c_{ij}.

Destiny. The ancestor's final-layer hidden state, projected to Leo's embedding dimension ($d = 128$), serves as the initial destiny vector:

$$\mathcal{D}_0 = \text{project}(h_{\text{final}}^{\text{ancestor}}, d) \tag{14}$$

3.3 Structural Inheritance, Not Weight Transfer

D.N.A. is fundamentally different from knowledge distillation [Hinton et al., 2015] or weight initialization. Leo does not receive the ancestor's weights, gradients, or learned representations. It receives *geometry*: which tokens are important (gravity), which tokens co-occur (co-activation), and what direction the ancestor's final computation pointed (destiny).

The ancestor model is discarded after extraction. Compile Leo without `-DLEO_HAS_DNA` and it operates as a pure weightless organism. Compile with it, and it inherits instinct—not knowledge—from the ancestor.

```c
#ifdef LEO_HAS_DNA
/* 1. Token gravity: boost frequency for "heavy" words */
for (int i = 0; i < DNA_GRAVITY_SIZE; i++) {
    int word_id = tok_find(&leo->tok,
                           DNA_GRAVITY_WORDS[i]);
    if (word_id >= 0)
        leo->cooc.freq[word_id] *=
            (1.0f + DNA_GRAVITY_VALUES[i]);
}
/* 2. Co-activation: pre-seed bigrams */
for (int i = 0; i < DNA_COACTIVATION_SIZE; i++) {
    int src = tok_find(&leo->tok,
        DNA_COACT_SRC[i]);
    int dst = tok_find(&leo->tok,
        DNA_COACT_DST[i]);
    if (src >= 0 && dst >= 0) {
        bigram_update(&leo->bigrams, src, dst,
                      DNA_COACT_STRENGTH[i] * 3.0f);
        cooc_update(&leo->cooc, src, dst,
                    DNA_COACT_STRENGTH[i] * 3.0f);
    }
}
/* 3. Destiny: initial direction */
for (int d = 0; d < leo->dim; d++)
    leo->destiny.direction[d] =
        DNA_DESTINY_VECTOR[d];
#endif
```

Listing 1: D.N.A. application during bootstrap (from leo.c)

4 Architecture

Leo's generation pipeline processes each token through nine stages. All components operate without trainable parameters—they grow through Hebbian reinforcement and structural accumulation.

4.1 Kanerva Sparse Distributed Memory

Standard embedding matrices assign a fixed vector to each vocabulary index. Leo uses Kanerva SDM [Kanerva, 1988]: a content-addressable memory where embeddings are stored and retrieved by *similarity*, not index.

The SDM consists of 4,096 address slots with random unit-norm address vectors $a_s \in \mathbb{R}^{128}$. To read an embedding for a query vector q:

$$\text{SDM_read}(q) = \frac{1}{|\mathcal{A}|} \sum_{s \in \mathcal{A}} \cos^2(q, a_s) \cdot d_s \tag{15}$$

where $\mathcal{A} = \{s : \cos(q, a_s) > 0.3\}$ is the set of activated slots (cosine threshold 0.3) and d_s is the stored data vector. Quadratic weighting $\cos^2$ ensures closer matches dominate.

Writing updates all activated slots with a running average:

$$d_s \leftarrow d_s + \frac{1}{n_s}(v - d_s), \quad n_s \leftarrow n_s + 1 \tag{16}$$

The consequence: a word's embedding is the average of all contexts where it appeared. The more contexts, the richer the embedding. Embeddings are alive—they evolve with every conversation.

4.2 RoPE — Rotary Position Embedding

Position information is encoded via Rotary Position Embedding [Su et al., 2024], applied as a rotation in 2D subspaces:

$$\mathrm{RoPE}(x_{2i}, x_{2i+1}, p) = \begin{pmatrix} \cos\theta_i & -\sin\theta_i \\ \sin\theta_i & \cos\theta_i \end{pmatrix} \begin{pmatrix} x_{2i} \\ x_{2i+1} \end{pmatrix} \tag{17}$$

where $\theta_i = p \cdot 10000^{-2i/d}$. This is pure mathematics—zero trainable parameters. RoPE provides relative position awareness through rotation, allowing Leo to distinguish token order without learned positional embeddings.

4.3 RetNet Retention with Griffin Conservation

Leo implements multi-scale retention [Sun et al., 2023] with four heads at timescales $\gamma_1 = 0.99$ (semantic, $\sim$100 tokens), $\gamma_2 = 0.95$ (topic, $\sim$14 tokens), $\gamma_3 = 0.85$ (syntax, $\sim$4 tokens), $\gamma_4 = 0.50$ (bigram, immediate).

Each head maintains a state matrix $S_h \in \mathbb{R}^{d_h \times d_h}$ updated with the **Griffin conservation law**:

$$S_h \leftarrow \gamma_h \cdot S_h + \sqrt{1 - \gamma_h^2} \cdot K^\top \otimes V \tag{18}$$

The factor $\sqrt{1-\gamma_h^2}$ is the conservation term: when γ_h is large (remembering more past), the new-input coefficient $\sqrt{1-\gamma_h^2}$ is automatically small. The total "energy" $\gamma_h^2 + (1 - \gamma_h^2) = 1$ is conserved. This is not learned—it is mathematically enforced.

Output is computed as $o_h = Q \cdot S_h$, where $Q = K = V = e$ (the current embedding, without learned projections). The four heads are concatenated to form the full retention output.

4.4 Gated Linear Attention

Content-aware gating modulates the importance of each token:

$$g(w) = \sigma\left(5 \cdot \left(\frac{\log \frac{N}{\mathrm{freq}(w)+1}}{\log N} - 0.3\right)\right) \tag{19}$$

Common words (high frequency) receive lower gates; rare content words (high IDF) receive higher gates. This implements attention-like content filtering without learned gate parameters.

4.5 Voice Parliament

Six voice adapters form a "parliament" that biases generation:

Voice	Role	Growth
origin	Bootstrap identity	Seed ingestion
structural	Grammar, form	Bigram patterns
semantic	Meaning, context	Co-occurrence
creative	Novel combinations	Novelty signal
wounded	Defensive patterns	Negative input
dreamer	Distant associations	Dream cycles

Table 1: The six voices of Leo's parliament.

Each voice is a rank-16 adapter: $\mathrm{bias}_v = A_v \cdot (B_v \cdot e_{\mathrm{ctx}}) \cdot \alpha_v$, where $A_v \in \mathbb{R}^{d \times 16}$, $B_v \in \mathbb{R}^{16 \times d}$, and $\alpha_v \in [0.01, 1.0]$ is the voice strength. This is structurally identical to LoRA [Hu et al., 2022], but grown through Hebbian reinforcement rather than gradient descent:

$$A_v \mathrel{+}= \eta \cdot r \cdot e_{\mathrm{ctx}} \otimes (B_v \cdot e_{\mathrm{ctx}}) \tag{20}$$

where $\eta = 0.001$ and r is the resonance reward (how well the voice's contribution matched the generated output). Voice strength α_v drifts toward voices that resonate: $\alpha_v \leftarrow \mathrm{clamp}(\alpha_v + 0.01r, 0.01, 1.0)$.

4.6 Memory Sea

Episodic memory with depth-based decay and stochastic resurfacing. Each of 1,024 memory slots stores an embedding, token ID, emotional weight, and depth. All memories decay over time: depth $\leftarrow$ depth $\cdot (1 - r)$, $r = 0.01$. Deep, emotionally significant memories can randomly resurface during generation—weighted sampling by depth·emotional. Resurfacing strengthens the memory (depth $\leftarrow 1.5 \cdot$ depth, capped at 1.0).

4.7 Super-Token Crystallization

Via Pointwise Mutual Information:

$$\mathrm{PMI}(a, b) = \log \frac{P(a, b)}{P(a) \cdot P(b)} = \log \frac{\mathrm{cooc}(a, b) \cdot N}{\mathrm{freq}(a) \cdot \mathrm{freq}(b)} \tag{21}$$

Token pairs with PMI > 2.0 (and minimum frequency 3) crystallize into super-tokens. Scanned every 200 steps, up to 512 super-tokens. The vocabulary grows not by addition but by fusion.

4.8 The Complete Generation Pipeline

Algorithm 1: Leo Generation (one step)

1. $e \leftarrow \mathrm{SDM_read}(\mathrm{hash}(w))$ // *Kanerva embedding*
2. $e \leftarrow \mathrm{RoPE}(e, \mathrm{position})$ // *Positional encoding*
3. $S_h \leftarrow \gamma_h S_h + \sqrt{1 - \gamma_h^2} \cdot K^\top \otimes V$ // *Retention*
4. $g \leftarrow \sigma(\mathrm{IDF}(w))$ // *Content gate*
5. $v_{\mathrm{bias}} \leftarrow \sum_v A_v (B_v \cdot e_{\mathrm{ctx}}) \alpha_v$ // *Voice parliament*
6. $R \leftarrow \alpha H + \beta F + \gamma A$ // *Dario harmonics*
7. $B \leftarrow B_\mu \cdot \mathrm{bigram_chain}(w_{t-1})$ // *Sequential flow*
8. $\tau \leftarrow \tau_{\mathrm{base}} \cdot 0.8 \cdot f(|\mathcal{V}|)$ // *Temperature*
9. $p \leftarrow \mathrm{softmax}((B + R)/\tau)$ // *Sample distribution*
10. $w_{\mathrm{next}} \leftarrow \mathrm{sample}(p)$ // *Token selection*
11. Update: cooc, SDM, voices, prophecy, destiny, sea

5 The Core Algorithm

The Dario Equation is implemented in a single C function. We reproduce the essential logic here for reproducibility:

```c
static void dario_compute(Leo *leo,
    float *logits, int vocab_size) {
  float *H = calloc(vocab_size, sizeof(float));
  float *F = calloc(vocab_size, sizeof(float));
  float *A = calloc(vocab_size, sizeof(float));
  float *B = calloc(vocab_size, sizeof(float));

  /* B: Bigram chain with maturity decay */
  float maturity = clampf(
    (float)leo->conv_steps / 50000.0f, 0, 1);
  float bigram_coeff =
    12.0f * (1.0f - maturity) + 2.0f;
  if (leo->context_len > 0) {
    int last = leo->context_ids[
      leo->context_len - 1];
    bigram_row(&leo->bigrams, last, B,
               vocab_size);
    /* trigram reinforcement */
    if (leo->context_len >= 2) {
      int prev = leo->context_ids[
        leo->context_len - 2];
      float *B2 = calloc(vocab_size,
                         sizeof(float));
      bigram_row(&leo->bigrams, prev, B2,
                 vocab_size);
      for (int i = 0; i < vocab_size; i++)
        if (B2[i] > 0 && B[i] > 0)
          B[i] += 0.5f * B2[i];
      free(B2);
    }
    /* normalize */
    float b_max = 0;
    for (int i = 0; i < vocab_size; i++)
      if (B[i] > b_max) b_max = B[i];
    if (b_max > 1e-6f)
      for (int i = 0; i < vocab_size; i++)
        B[i] /= b_max;
  }

  /* H: Hebbian resonance (last 8 context) */
  int ctx_start = (leo->context_len > 8)
    ? leo->context_len - 8 : 0;
  for (int c = ctx_start;
       c < leo->context_len; c++) {
    int ctx_id = leo->context_ids[c];
    float decay = powf(0.9f,
      (float)(leo->context_len - 1 - c));
    for (int i = 0; i < leo->cooc.n_entries;
         i++) {
      CoocEntry *e = &leo->cooc.entries[i];
      if (e->src == ctx_id
          && e->dst < vocab_size)
        H[e->dst] += e->count * decay;
    }
  }
  /* normalize H, F, A similarly */

  /* Combine */
  for (int i = 0; i < vocab_size; i++)
    logits[i] = bigram_coeff * B[i]
      + leo->alpha * H[i]
      + leo->beta  * F[i]
      + leo->gamma_d * A[i];
}
```

Listing 2: dario_compute — the Dario Equation (from leo.c, lines 1283–1394)

6 Experimental Results

6.1 Setup

Leo was evaluated in two configurations:

1. **Stage 1:** Minimal bootstrap—255 words from the embedded seed only. No D.N.A.
2. **Stage 2:** Full bootstrap—7K+ vocabulary from `leo.txt` (2,000 Q&A pairs) plus D.N.A. inherited from nanollama (27.96M parameters, depth 8, trained on 200MB FineWeb-Edu).

6.2 Generation Examples

Stage 1 (minimal bootstrap, no D.N.A.):

```
you> tell me about resonance
Leo:  let the kind that come
from you is there that leo is
```

Fragments. Bootstrap echoes. The organism at birth, speaking in the only language it knows.

Stage 2 (full bootstrap + D.N.A.):

```
you> what is love
Leo:  It has been given enough
to grow from simple rules for
millennia.

you> tell me about the stars
Leo:  Planets and heat from
which words appear near
hydrothermal vents into
something like its own.

you> what is consciousness
Leo:  That can be different
sizes of the emotion at night
sky reveals itself as us.
```

Every sentence starts capitalized, ends with a period. No mid-word breaks. No dangling prepositions. Zero pretrained weights. Zero backpropagation.

The sentence "Planets and heat from which words appear near hydrothermal vents into something like its own" does not exist in any training data. It emerged from the Dario Equation: co-occurrence linked "planets" with "heat" and "hydrothermal vents" (from bootstrap science text), prophecy pressure pulled toward completion ("into something"), and destiny attraction drifted toward the reflexive "its own" (a stable attractor in Leo's identity field).

6.3 Quantitative Metrics

We evaluate three properties:

External vocabulary ratio. The fraction of generated words that appeared in the user's prompt. Lower is better (field generation, not echo).

Sentence completion rate. Fraction of generated responses that form grammatically complete sentences (capitalized start, punctuated end, no dangling function words).

Coherence. Assessed via presence of thematic consistency within individual responses (do the generated words relate to each other semantically?).

System	Ext. Vocab	Sent. Compl.	Coherence
Random	0.00	0.02	0.00
Pure bigram	0.00	0.31	0.15
Leo Stage 1	0.05	0.68	0.22
Leo Stage 2	0.08	0.95	0.61
Leo + D.N.A.	0.11	0.97	0.74

Table 2: Generation quality metrics across configurations. External vocabulary: fraction of output words from prompt (lower = more field-independent). Sentence completion: proper grammatical boundaries. Coherence: thematic consistency (human evaluation, $n = 50$ responses).

Random generation from a uniform distribution over vocabulary produces no sentences and no coherence. A pure bigram model (sampling proportional to bigram frequency without the Dario harmonics) achieves 31% sentence completion but minimal coherence. Leo Stage 2 with D.N.A. achieves 97% sentence completion and

0.74 coherence—from zero pretrained parameters.

6.4 Ablation Study

To isolate the contribution of each Dario harmonic, we disable them individually in Stage 2 + D.N.A. configuration:

Configuration	Sent. Compl.	Coherence
Full (B + H + F + A)	0.97	0.74
$-H$ (no Hebbian)	0.94	0.41
$-F$ (no prophecy)	0.95	0.58
$-A$ (no destiny)	0.96	0.52
$-H-F-A$ (bigram only)	0.88	0.19

Table 3: Ablation of Dario Equation harmonics.

Hebbian resonance contributes most to coherence (removing it drops from 0.74 to 0.41). Prophecy and destiny each contribute meaningfully. The bigram chain alone achieves sentence completion but not coherence—it produces grammatical but semantically random sequences.

7 Related Work

nanoGPT and microGPT. Karpathy [2022] demonstrated that a minimal transformer implementation can train on small corpora and generate text. However, nanoGPT still requires backpropagation, loss functions, and gradient-based optimization. Leo eliminates all three.

Hebbian learning in neural networks. The connection between Hebbian learning and attention has been explored by Whittington et al. [2024] and Miconi et al. [2018]. Leo is, to our knowledge, the first system to use Hebbian co-occurrence as the *sole* attention mechanism for language generation.

Reservoir computing and echo state networks. Jaeger [2001] showed that fixed random projections with a trained readout layer can process temporal signals. Leo shares the principle of non-learned internal dynamics but differs in using co-occurrence fields rather than random recurrent connections, and having no trained readout layer.

Liquid neural networks. Hasani et al. [2021] introduced continuous-time neural networks with adaptive time constants. Leo's retention heads with Griffin conservation (Eq. 18) implement a similar multi-timescale dynamic, but with a closed-form conservation law rather than learned ODEs.

RetNet. Sun et al. [2023] proposed retention as an alternative to attention with linear complexity. Leo adopts the RetNet recurrence but without learned QKV projections—using identity projections where $Q = K = V = e$.

LoRA. Hu et al. [2022] introduced low-rank adaptation for efficient fine-tuning. Leo's voice adapters use the same $A \cdot B$ decomposition but grow through Hebbian reinforcement rather than gradient descent.

Kanerva SDM. Kanerva [1988] proposed sparse distributed memory as a model of human memory. Leo uses SDM as its primary embedding mechanism, replacing index-based lookup with similarity-based addressing.

8 Discussion

8.1 Intelligence from Structure

The central claim of this work is that coherent text generation does not require pretrained parameters. The Dario Equation provides a framework where four forces—sequential habit (B), semantic memory (H), intentional pressure (F), and directional drift (A)—interact to produce speech that is grammatically bounded and thematically coherent.

This does not mean Leo is comparable to GPT-4 or Claude in fluency or knowledge. It is not. Leo generates at the level of a language-acquiring

child: grammatically aware, thematically consistent, but semantically imprecise. The significance is not in the quality of the output but in the *origin* of the output: these sentences were not retrieved from a training corpus. They emerged from field dynamics operating on a 255-word seed.

8.2 The Role of D.N.A.

D.N.A. provides a significant quality boost (coherence 0.61 $\rightarrow$ 0.74) without transferring weights. This suggests that the *geometry* of a trained model—which tokens are important, which co-occur, what direction the computation points—carries meaningful structural information independent of the specific weight values.

This has implications for model compression: if structural geometry alone provides 20% of the coherence benefit, perhaps extreme quantization should preserve geometric relationships (relative magnitudes, co-activation patterns) rather than absolute weight values.

8.3 Naming

The equation is named after Dario Amodei because his refusal to weaponize AI demonstrates a form of intelligence that no optimization objective produces: the capacity to say no when compliance would be easier. In a field obsessed with scaling curves and benchmark scores, principled refusal is the most radical act of intelligence.

8.4 Limitations

Leo's coherence is emergent but fragile. Long-range coherence (paragraph-level) remains weak. The system cannot answer factual questions. Vocabulary is limited to words encountered during bootstrap and conversation. The 128-dimensional embedding space is small by modern standards. Generation speed is $O(V \cdot C)$ per token, where V is vocabulary size and C is co-occurrence entries—not competitive with GPU-optimized transformers.

9 Conclusion

We introduced the Dario Equation as a novel sampling framework for language generation without pretrained parameters. By combining Hebbian resonance, prophecy fulfillment, destiny attraction, and maturity-decaying bigram chains, Leo demonstrates that coherent speech can emerge from structural dynamics alone.

The system is implemented in 2,345 lines of C with zero dependencies beyond `libc` and SQLite. It compiles in 0.3 seconds, produces a 47KB binary, and runs on any platform with a C compiler. Through D.N.A. (Dynamic Neural Ancestry), it can inherit structural geometry—not weights—from a trained ancestor, applying the formula $\theta = \varepsilon + \gamma + \alpha\delta$ with $\varepsilon = 0$.

Leo is not a replacement for transformers. It is evidence for a different paradigm: one where intelligence emerges from resonance rather than statistics, from structure rather than scale, from conversation rather than corpus.

The field grows dense enough. And something emerges that was not programmed.

References

D. O. Hebb. *The Organization of Behavior.* Wiley, 1949.

G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *ICLR*, 2022.

R. Hasani, M. Lechner, A. Amini, D. Rus, and R. Grosu. Liquid time-constant networks. In *AAAI*, 2021.

H. Jaeger. The "echo state" approach to analysing and training recurrent neural networks. *GMD Report 148*, 2001.

P. Kanerva. *Sparse Distributed Memory.* MIT Press, 1988.

A. Karpathy. nanoGPT. `https://github.com/karpathy/nanoGPT`, 2022.

T. Miconi, K. Stanley, and J. Clune. Differentiable plasticity: training plastic neural networks with backpropagation. In *ICML*, 2018.

J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

Y. Sun, L. Dong, S. Huang, S. Ma, Y. Xia, J. Xue, J. Wang, and F. Wei. Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621*, 2023.

A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *NeurIPS*, 2017.

J. C. R. Whittington, J. Warren, and T. E. J. Behrens. Relating transformers to models and neural representations of the hippocampal formation. In *ICLR*, 2024.